Semester project on control and simulation of automation systems

BEng in Robot Systems

4. Semester

Authors (Group 1?)

Name	Username	Date of birth
Noah V. Vryens	novry24	01/04/2002
Rasmus K. Nielsen	rasni19	25/12/1997
Emma B. Rasmussen	erasm24	15/05/2001
Jannick H. Irvold	jairv24	23/10/2002
Eymundur Ó. Pálsson	eypal24	31/08/2002

Supervisor

Cheng Fang

chfa@mmmi.sdu.dk

Total characters:

May 26, 2026

hii, im an abstract :3

Contents

1 Introduction

In life and industri that are alot of machinery that need regualted and precice controll, which i why a controller is made to achive each goal as needed. In robotics controllers can be expicaly usefull when trying to complete difficult taskes with multipul moveing parts. Where balance is one of main hindrences, when trying to design such a system. This projeckt will focus trying to solve a balance isuee of an inverted pendulem.

1.1 Project Goals

With a PLC, motor, conveyor track, inverted pendulem on a cart and a two diffrencet controller. it is attempted to make the inverted pendulem balance rightsideup using a controller to controll the force of the motor. success is mesurd by how well the controller balences the pendulem compared to a simulation of the controller, and the best controller is the one whose closest to the simulation of said controller.

1.2 Problem Definition

1.3 Constraints & Limitations

The limitations for this projeckt come from the physical setup, and the PLC. The cart is limited to move within two sensors on the conveyor track, and the maximum force for the moter is X. For the PLC it cant run faster than 1000Hz, and sampling via OPCUA is limited to 100Hz

1.4 System OverView

2 System modeling

2.1 Modelling of the Inverted Pendulum System

2.1.1 System Overview

The purpose of the modelling process is to obtain a mathematical representation of the inverted pendulum system, which can later be used for simulation and controller design. A mathematical model makes it possible to analyze the dynamic behavior of the system and evaluate different control strategies before implementation on the physical setup.

In this project, two classical modelling approaches were used: the Newton-Euler method and the Euler-Lagrange method. Both methods describe the dynamics of the system, but from different physical perspectives. The Newton-Euler method is based on force and moment balances, while the Euler-Lagrange method is based on the kinetic and potential energy of the system. The resulting nonlinear equations were linearized around the upright equilibrium position to simplify the controller design and implementation in MATLAB/Simulink.

The physical system consists of a cart with an attached inverted pendulum and an additional tip mass mounted at the end of the pendulum rod. The cart is constrained to move horizontally along a conveyor-driven rail, while the pendulum is free to rotate around its pivot point. The system is actuated by an external force F, which controls the horizontal movement of the cart. The goal of the system is to keep the pendulum balanced in its upright position while controlling the motion of the cart.

The generalized coordinates used to describe the system are:

- x: horizontal cart position
- θ: pendulum angle relative to the vertical axis

The physical parameters used in the model are shown in Table **??**.

The following assumptions were made during the modelling process:

- The pendulum rod is considered rigid
- Friction is assumed to be small and primarily represented by damping coefficients
- The pendulum motion is constrained to a two-dimensional plane
- The system is linearized around the upright equilibrium position

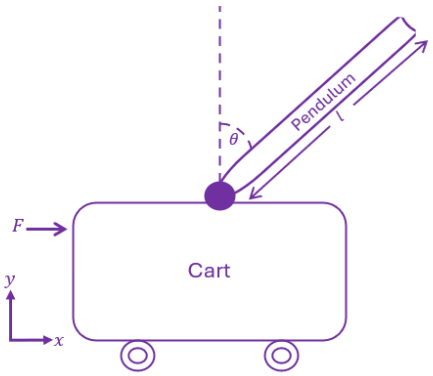

Figure 2.1: Simplified model of the inverted pendulum system showing the generalized coordinates, pendulum angle, and input force.

Table 2.1: System parameters used for modelling of the inverted pendulum system

Symbol	Parameter	Value	Unit
M	Mass of cart	0.500	kg
m	Mass of pendulum (including tip mass)	0.084	kg
l	Length of pendulum rod	0.35	m
L_{belt}	Length of conveyor belt	1.72	m
r	Radius of belt rollers	0.05	m
g	Gravitational acceleration	9.82	m/s^2
b_{lin}	Linear damping coefficient	5	N/(m/s)
b_{rot}	Rotational damping coefficient	0.0012	Nm/(rad/s)

2.1.2 Newton-Euler Modelling

The Newton-Euler method is based on applying Newton's second law and rotational moment equations to the system. Free body diagrams were constructed for both the cart and the pendulum to identify the forces and moments acting on the system.

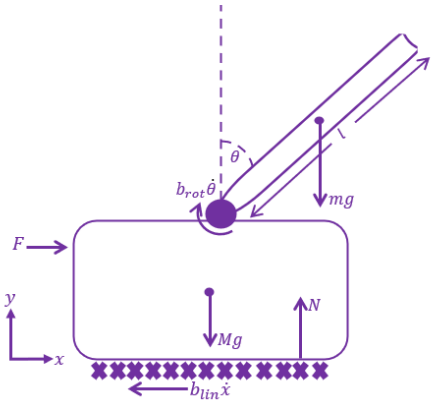

Figure 2.2: Free body diagram of the inverted pendulum system showing the applied force, gravitational forces, damping effects, and normal force acting on the system.

For the cart, the sum of forces in the horizontal direction was established using Newton's second law:

$$\sum F_x = M\ddot{x} \tag{2.1}$$

For the pendulum, both translational and rotational dynamics were considered. The pendulum experiences gravitational forces, reaction forces from the cart, and inertial effects caused by the cart motion.

To eliminate the internal reaction forces between the cart and pendulum, the moment equation was derived around the pendulum center of mass. This resulted in a coupled set of nonlinear differential equations describing the dynamic behavior of the system.

The obtained equations contain nonlinear trigonometric terms originating from the pendulum rotation. To simplify the controller design, the equations were linearized around the upright equilibrium position using the small-angle approximations:

$$\sin(\theta) \approx \theta \tag{2.2}$$

$$\cos(\theta) \approx 1 \tag{2.3}$$

After linearization, the system equations were transformed into transfer functions and state-space representations suitable for simulation and controller design in MATLAB/Python.

Final Linearized Model

$$(I + ml^2)\ddot{\theta} - mgl\theta = ml\ddot{x} \tag{2.4}$$

$$(M + m)\ddot{x} + b\dot{x} - ml\ddot{\theta} = u \tag{2.5}$$

State-Space Representation

$$\dot{x} = Ax + Bu \tag{2.6}$$

$$y = Cx + Du \tag{2.7}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-b(I+ml^2)}{(M+m)I+Mml^2} & \frac{m^2gl^2}{(M+m)I+Mml^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-mlb}{(M+m)I+Mml^2} & \frac{mgl(M+m)}{(M+m)I+Mml^2} & 0 \end{bmatrix} \tag{2.8}$$

$$B = \begin{bmatrix} 0 \\ \frac{I+ml^2}{(M+m)I+Mml^2} \\ 0 \\ \frac{ml}{(M+m)I+Mml^2} \end{bmatrix} \tag{2.9}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2.10}$$

$$D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{2.11}$$

2.1.3 Euler-Lagrange Modelling

The Euler-Lagrange method describes the system dynamics using energy relationships instead of direct force balances. The method is based on the kinetic and potential energy of the system.

The kinetic energy of the cart is given by:

$$T_{cart} = \frac{1}{2}M\dot{x}^2 \tag{2.12}$$

The pendulum contributes both translational and rotational kinetic energy due to its motion relative to the cart. Additionally, the pendulum possesses potential energy caused by gravity.

Using the derived expressions for kinetic and potential energy, the Lagrangian was formulated as:

$$L = T - V \tag{2.13}$$

The equations of motion were then derived using the Euler-Lagrange equation:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = Q_i \tag{2.14}$$

where q_i represents the generalized coordinates of the system.

By applying the Euler-Lagrange formulation to both the cart position and pendulum angle, a coupled nonlinear dynamic model of the system was obtained.

Similar to the Newton-Euler approach, the nonlinear equations were linearized around the upright equilibrium position to simplify further analysis and controller design.

Kinetic and Potential Energy

$$E_{kin} = E_{kin,c} + E_{kin,p} + E_{kin,rot} \tag{2.15}$$

$$= \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m\dot{x}^2 - ml\dot{\theta}\dot{x}\cos\theta + \frac{1}{2}ml^2\dot{\theta}^2 + \frac{1}{2}I\dot{\theta}^2 \tag{2.16}$$

$$E_{pot} = mgl\cos\theta \tag{2.17}$$

Nonlinear Equations of Motion

$$-ml\ddot{x}\cos\theta + ml^2\ddot{\theta} + I\ddot{\theta} - mlg\sin\theta = -b_{rot}\dot{\theta} \tag{2.18}$$

$$(M+m)\ddot{x} - ml\ddot{\theta}\cos\theta + ml\dot{\theta}^2\sin\theta = F - b_{lin}\dot{x} \tag{2.19}$$

Linearized Equations

$$ml^2\ddot{\theta} + I\ddot{\theta} - mlg\theta + b_{rot}\dot{\theta} = ml\ddot{x} \tag{2.20}$$

$$(M+m)\ddot{x} - ml\ddot{\theta} + b_{lin}\dot{x} = u \tag{2.21}$$

Transfer Function

$$\frac{\Theta(s)}{U(s)} = \frac{1}{\left((M+m)\cdot\frac{ml^2s^2+Is^2-mlg+b_{rot}s}{mls^2}\right)s^2 - mls^2 + b_{lin}\left(\frac{ml^2s^2+Is^2-mlg+b_{rot}s}{mls^2}\right)s} \tag{2.22}$$

2.1.4 Comparison of the Methods

Both modelling approaches resulted in equivalent dynamic models after linearization, verifying the correctness of the derivations.

The Newton-Euler method provided a more intuitive understanding of the physical forces and moments acting on the system, since it directly uses force and torque balances. However, the method becomes increasingly complex for systems with multiple interconnected bodies.

The Euler-Lagrange method provided a more systematic and compact mathematical formulation by describing the system through energy relationships. This makes the method particularly useful for more advanced multibody systems.

The final linearized model obtained from the derivations was used as the foundation for the controller design and simulations presented later in this project.

The final linearized model was implemented in MATLAB/Python and used for simulation and controller development. Both transfer function and state-space representations were used throughout the project for analysis and validation of the implemented controllers.

The derived model forms the basis for the PID controllers, pole-placement controller, and the discrete implementation used later in the project.

3 Control theroy

3.1 LQR controller

One way to stabilize the inverted pendulum in its upward position, is a Linear Quadratic Regulator (LQR). LQR is an "optimal" multivariable controller that determines the best possible feedback gains by balancing the trade off between keeping the system states close to zero and minimizing the control effort used to do so.

3.1.1 Discretization for PLC

Because the controller will be running on a B&R Programmable Logic Controller (PLC) running at a fixed cycle time of $T_s = 0.001$ seconds, the continuous time statespace model cannot be used directly.
The system matrices are first discretized using a Zero-Order Hold (ZOH) method. This converts the physical model into a discrete-time representation that aligns with the digital sampling of the PLC.

3.1.2 The Cost Function and Weighting Matrices

The discrete LQR algorithm computes an optimal gain matrix K by minimizing the discrete quadratic cost function:

$$J = \sum_{k=0}^{\infty} \left(x_k^T Q x_k + u_k^T R u_k \right) \tag{3.1}$$

Where:

- x_k is the state vector at time step k, defined as $x = [x_c, \dot{x}_c, \theta, \dot{\theta}]^T$.
- u_k is the control input (motor force).
- Q is a matrix that penalizes deviations in the system states (position and velocity errors).
- R is a matrix that penalizes the use of the actuator (control effort).

3.1.3 Tuning with Bryson's Rule

Deciding what to put in the Q and R matrices in an intuitive way, Bryson's Rule is applied. This method scales each parameter based on its maximum acceptable physical limit, ensuring every variable contributes evenly to the cost function.

The maximum limits defined for this system are:

- **Cart Position (x_c):** 0.5 m
- **Cart Velocity ($\dot{x}_c$):** 1.0 m/s
- **Pendulum Angle (θ):** $\frac{\pi}{4}$ rad
- **Pendulum Velocity ($\dot{\theta}$):** 5.0 rad/s

- **Actuator Force (F):** 14.4 N

Using these limits, the diagonal values of the Q and R matrices are calculated as the inverse square of the maximum acceptable values ($\frac{1}{\max^2}$).

3.1.4 The Control Law

Solving the algebraic Riccati equation with these Q and R weights yields the optimal feedback gain matrix K. In the PLC implementation, the control force applied to the cart is calculated simply by multiplying the static gain matrix by the current sensor states:

$$u = -Kx = -(K_1 x_c + K_2 \dot{x}_c + K_3 \theta + K_4 \dot{\theta}) \tag{3.2}$$

3.2 LQI controller

While standard LQR is very effective at stabilizing the system, it functions purely as a proportional controller. This means it can suffer from steady-state errors if there are unmodeled dynamics or constant disturbances in the system, such as friction in the belt which the cart sits on or other forces affecting the cart or pendulum.

3.2.1 Augmenting the State Vector

To eliminate these steady state offsets, the controller can be expanded into a Linear Quadratic Integral (LQI) controller. This is achieved by introducing a new state variable that accumulates the cart position error over time, functioning like the "I" term in a PID controller.

This new integral state is defined as:

$$x_i = \int_0^t (x_{ref} - x_c) d\tau \tag{3.3}$$

The standard state vector is then augmented from four variables to five:

$$x_{aug} = \left[x_c, \dot{x}_c, \theta, \dot{\theta}, x_i\right]^T \tag{3.4}$$

3.2.2 The Augmented Control Law

To implement LQI, the discrete system matrices (A and B) and the penalty matrices (Q and R) are expanded to fit this fifth state. The augmented Q matrix receives a new diagonal value specifically to penalize the integral error.

Solving the algebraic Riccati equation again for this augmented system gives the new gain matrix, and the resulting control law becomes:

$$u = -K_{aug} x_{aug} = -(K_1 x_c + K_2 \dot{x}_c + K_3 \theta + K_4 \dot{\theta}) - K_i x_i \tag{3.5}$$

Because the $K_i x_i$ term continuously builds up control effort as long as a there is an error in the cart position, it guarantees that the cart will settle on its target position, regardless of small physical imperfections or friction.

3.3 Cascaded PID controller

While modern statespace methods like LQR are very effective for multivariable systems, classical control techniques can also be applied. Since regulating both the cart position and the pendulum position is required to balance the pendulum on the limited belt length, a single controller will not be enough. Instead, two cascaded regulators is used, one being a Proportional-Derivative (PD) controller for the pendulum and the other, a Proportional-Integral-Derivative (PID) controller for the cart.

3.3.1 Cascade setup

The control system is divided into two nested feedback loops:

- **Inner Loop (Pendulum):** This loop focuses entirely on stabilizing the pendulum angle (θ). It uses a PD controller. The integral term is not used here, as it can introduce phase lag and worsen the stability of the highly unstable system of the inverted pendulum. The derivative term provides the critical damping required to catch and balance the pendulum.
- **Outer Loop (Cart):** This loop controls the cart's position (x_c) using a full PID controller. Instead of outputting force directly, it calculates the necessary pendulum angle (θ_{ref}) required to move the cart. The integral term is used in this outer loop to eliminate steady state positional errors caused by track friction or unmodeled slight inclinations in the setup.

For this cascade to function, the inner angle loop must be tuned to react significantly faster than the outer position loop, preventing the two controllers from fighting each other.

3.3.2 Tuning via Root Locus

The primary method used for tuning these controllers is the Root Locus technique, which visualizes the paths of the closed loop poles as a control gain is varied.

The transfer function for the inner PD controller is:

$$C_{PD}(s) = K_{p,\theta} + K_{d,\theta}s \tag{3.6}$$

This structure adds a single zero to the open loop system. The inner loop is tuned first by strategically placing this zero on the s-plane to "pull" the unstable open loop poles of the pendulum into the stable left-half plane (LHP), ensuring the system meets the performance requirements (write more on requirements here and on LQR!).

Once the inner loop is closed and verified to meet the goals, the outer loop is tuned. The transfer function for the outer PID controller is:

$$C_{PID}(s) = K_{p,x} + \frac{K_{i,x}}{s} + K_{d,x}s = \frac{K_{d,x}s^2 + K_{p,x}s + K_{i,x}}{s} \tag{3.7}$$

The PID controller introduces one pole at the origin (from the integrator) and two zeros. Using Root Locus on the new combined system, these zeros are placed to change the trajectory of the cart's poles. The goal here is to target ensure the cart moves slowly so that the pendulum is not destabilzed by aggressive changes to the pendulum angle (θ_{ref}).

3.3.3 Alternative Tuning Methods

While Root Locus provides an intuitive, graphical understanding of system stability, several other tuning methodologies exist to refine the initial gains:

- **Frequency Response (Bode/Nyquist Plots):** This method shapes the open loop frequency response to achieve specific gain and phase margins. It is particularly useful for ensuring the system remains stable despite unmodeled high frequency dynamics (like sensor noise or actuator lag) and for precisely decoupling the bandwidths of the inner and outer loops.
- **Computational Optimization:** Software tools (such as MATLAB's `pidtune`) can algorithmically calculate gains by minimizing a specified cost function, such as the Integral Time Absolute Error (ITAE). This approach systematically balances rise time, settling time, and overshoot against a mathematically defined performance metric.

3.4 Swing up

Something that is not nescessary for balancing the pendulum itself is swing up, instead the purpose of this is to get the pendulum from hanging downward to within a specific range of the upward position where a controller can catch it. This makes the testing process of the different controllers much easier and the overall implementation and use of the system simpler.

3.4.1 The Energy Controller

To achieve the swing up, the system uses an energy based controller. This injects kinetic energy into the pendulum until its total mechanical energy equals the potential energy required to stand upright.

The total mechanical energy (E) of the pendulum is calculated as the sum of its kinetic and potential energy:

$$E = \frac{1}{2} I_p \dot{\theta}_{filtered}^2 + m_p g l(\cos\theta - 1) \tag{3.8}$$

The system defines the upward position as having an energy state of exactly 0.0.

By subtracting 1.0 from the cosine of theta, the potential energy is offset so that it peaks at zero when the pendulum is in the upward position. The controller then calculates the energy error as the difference between this target state (0.0) and the pendulum's current mechanical energy:

$$E_{error} = 0.0 - E \tag{3.9}$$

To put more energy into the pendulum, the controller must accelerate the cart in step with the pendulum's swing. The direction of this acceleration is determined by the sign of the product of the filtered angular velocity and the cosine of theta ($\dot{\theta}_{filtered}\cos\theta$).

3.4.2 The Control Law & Exit Condition

The control effort (u) is then calculated using the energy error, an energy gain constant, and the direction (d). However, just putting in energy to the pendulum could cause the cart to drift so a small spring force is added to gently center the cart:

$$u = k_{energy} E_{error} \cdot \text{direction} + k_x x \tag{3.10}$$

The swing up PLC state always monitors the pendulum's position and velocity to figure out if it is safe to hand over to a controller. This "catch condition" triggers only when two conditions are met at the same time:

The position is within $\pm 30^\circ$ of the upward position ($|\theta| < \frac{\pi}{6}$ rad).
The velocity is low enough for a controller to catch ($|\dot{\theta}| < 1$ rad/s).

Once the conditions are met all errors and variables from controllers are reset and the PLC state is switched to controller to balance the pendulum.

4 PLC Implementation

Moving from a simulated environment to a physical one comes with a list of challenges that need to solved. Firstly, control of the cart must be understood. The cart is driven along a rail by a belt, attached to a DC motor. The motor is connected to a motor controller which in turn is connected to the PLC via an analog signal. The motor controller can be set to a target torque mode, which aims to hit a percentage of the maximum torque of the motor based on the analog input. With this information, a scaling factor can be used when the PLC needs to calculate how much voltage needs to be send via the analog signal to apply a certain torque in newton on to the cart. The scaling factor is found as follows based on the resolution of the analog signal (16 bit integer), belt's roller diameter [meter] and motor's max torque [newton meter]:

$$scalingFactor = \left\lfloor \frac{analogResolution \cdot rollerDiameter}{maxTorque} \right\rfloor = \left\lfloor \frac{32,768 \cdot 0.05}{0.72} \right\rfloor = 2,275 \tag{4.1}$$

This also allows for the calculation of the maximum possible force that can be applied to the cart:

$$maxForce = \frac{maxTorque}{rollerDiameter} = \frac{0.72}{0.05} = 14.4[N] \tag{4.2}$$

In order to interact with the controllers, the inputs from the encoders must be converted. The encoder for the pendulum has a resolution of 4096 units per full cycle and is reset such that the encoder shows 0 when the pendulum is hanging straight down. The controller expects the input to be in radians with 0 pointing straight up. Thus the conversion can be done as follows:

$$theta[rad] = ((encoder_{pendulum} \bmod 4096) - 2048) \cdot \left(\frac{\pi \cdot 2}{4096} \right) \tag{4.3}$$

The conversion for the cart position was found using a measured distance. Moving the cart from one end of the track to the other resulted in the encoder showing 381,709 units while the cart moved 156.7 centimeters. Thus the amount of units per meter can be found as:

$$unitsPerMeter = \frac{381,709}{1.567} = 243,592 \tag{4.4}$$

The controllers expect the input to be in meters, with 0 at the center of the track, which is set to 191,000 units, roughly half of the full length. The conversion looks as follows:

$$x[m] = \frac{encoder_{cart} - 191,000}{243,592} \tag{4.5}$$

5 Data Collection

6 Evaluation

7 Discussion

8 Conclusion

References

A Distribution of Tasks

Table A.1: Distribution of Tasks

Task	Noah	Rasmus	Emma	Jannick	Eymundur
Development					
System Modeling					
Control Theroy					
PLC implementation					
Data Collection					
Report					
Abstract					
Introduction					
System Modeling					
Control theroy					
PLC Implementation					
Data Collection					
Results					
Discussion					
Conclusion					

B Code

The code is available in the attached ZIP file as well as on github at:

C Video Showcase

A video of the system running is available at: